Fixing Rock-Paper-Scissors Outcome Logic

https://gemini.google.com/app/bdcd212edc8adb97

*User prompt: Here is a broken Rock-Paper-Scissors Python script: [paste code]. When run, it causes Whenever the player throws a winning hand against the computer, the game awards the point to the CPU instead of the player. Correct the outcome evaluation logic so standard rules apply (Rock beats Scissors, Scissors beats Paper, and Paper beats Rock).. Please diagnose the bug and fix it while keeping the existing structure clean.import random import pygame from game.button import ChoiceButton class GameEngine: def __init__(self, width, height): self.width = width self.height = height self.choices = ["ROCK", "PAPER", "SCISSORS"] btn_w, btn_h = 130, 50 gap = 20 total_w = 3 * btn_w + 2 * gap start_x = (width - total_w) // 2 btn_y = height - 85 self.buttons = [ChoiceButton("ROCK", pygame.Rect(start_x, btn_y, btn_w, btn_h), (160, 50, 50), (200, 70, 70)), ChoiceButton("PAPER", pygame.Rect(start_x + btn_w + gap, btn_y, btn_w, btn_h), (40, 100, 170), (60, 130, 210)), ChoiceButton("SCISSORS", pygame.Rect(start_x + 2 * (btn_w + gap), btn_y, btn_w, btn_h), (180, 140, 30), (220, 180, 50)),] self.player_choice = None self.cpu_choice = None self.result_text = "Make your move!" self.result_color = (220, 225, 235) self.player_score = 0 self.cpu_score = 0 self.round_resolved_time = 0 self.display_duration = 1800 self.showing_result = False self.font_title = pygame.font.SysFont(None, 36) self.font_hud = pygame.font.SysFont(None, 26) self.font_arena = pygame.font.SysFont(None, 32) def determine_winner(self, player, cpu): if player == cpu: return "TIE" rules = { ("ROCK", "SCISSORS"): "CPU", ("SCISSORS", "PAPER"): "CPU", ("PAPER", "ROCK"): "CPU", ("SCISSORS", "ROCK"): "PLAYER", ("PAPER", "SCISSORS"): "PLAYER", ("ROCK", "PAPER"): "PLAYER", } return rules.get((player, cpu), "TIE") def play_round(self, choice): self.player_choice = choice self.cpu_choice = random.choice(self.choices) outcome = self.determine_winner(self.player_choice, self.cpu_choice) if outcome == "PLAYER": self.player_score += 1 self.result_text = f"You Win! {self.player_choice} beats {self.cpu_choice}." self.result_color = (80, 230, 120) elif outcome == "CPU": self.cpu_score += 1 self.result_text = f"You Lose! {self.cpu_choice} beats {self.player_choice}." self.result_color = (240, 80, 80) else: self.result_text = f"It's a Draw! Both picked {self.player_choice}." self.result_color = (240, 210, 80) self.showing_result = True self.round_resolved_time = pygame.time.get_ticks() def handle_event(self, event): if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1: for btn in self.buttons: if btn.contains(event.pos): self.play_round(btn.choice_name) break def update(self): now = pygame.time.get_ticks() if self.showing_result and (now - self.round_resolved_time >= self.display_duration): self.player_choice = None self.cpu_choice = None self.result_text = "Make your move!" self.result_color = (190, 195, 205) self.showing_result = False def render(self, screen): screen.fill((24, 28, 36)) title_surf = self.font_title.render("Rock Paper Scissors", True, (245, 245, 245)) screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 14)) p_surf = self.font_hud.render(f"Player Score: {self.player_score}", True, (100, 180, 255)) c_surf = self.font_hud.render(f"CPU Score: {self.cpu_score}", True, (255, 120, 120)) screen.blit(p_surf, (35, 52)) screen.blit(c_surf, (self.width - c_surf.get_width() - 35, 52)) pygame.draw.line(screen, (45, 52, 66), (25, 82), (self.width - 25, 82), 2) p_str = self.player_choice if self.player_choice else "--" c_str = self.cpu_choice if self.cpu_choice else "--" arena_p = self.font_arena.render(f"Your Pick: {p_str}", True, (225, 225, 230)) arena_c = self.font_arena.render(f"CPU Pick: {c_str}", True, (225, 225, 230)) screen.blit(arena_p, (self.width // 2 - arena_p.get_width() // 2, 115)) screen.blit(arena_c, (self.width // 2 - arena_c.get_width() // 2, 155)) res_surf = self.font_arena.render(self.result_text, True, self.result_color) screen.blit(res_surf, (self.width // 2 - res_surf.get_width() // 2, 205)) for btn in self.buttons: btn.render(screen)*

Response: The bug is located in the `rules` dictionary inside `determine_winner()`. The outcomes are inverted: player-winning matchups like `("ROCK", "SCISSORS")` are mapped to `"CPU"`, while player-losing matchups like `("SCISSORS", "ROCK")` are mapped to `"PLAYER"`.

The Diagnosis

In your current code:

```
rules = {
    ("ROCK", "SCISSORS"): "CPU",      # Bug: Player (Rock) beats CPU (Scissors), but awards point to CPU
    ("SCISSORS", "PAPER"): "CPU",     # Bug: Player (Scissors) beats CPU (Paper), but awards point to CPU
    ("PAPER", "ROCK"): "CPU",         # Bug: Player (Paper) beats CPU (Rock), but awards point to CPU
    ("SCISSORS", "ROCK"): "PLAYER",   # Bug: CPU (Rock) beats Player (Scissors), but awards point to Player
    ("PAPER", "SCISSORS"): "PLAYER",  # Bug: CPU (Scissors) beats Player (Paper), but awards point to Player
    ("ROCK", "PAPER"): "PLAYER",      # Bug: CPU (Paper) beats Player (Rock), but awards point to Player
}
```

Since the tuple key represents `(player, cpu)`, mapping `("ROCK", "SCISSORS")` to `"CPU"` gives the win to the computer whenever the player throws the winning move.

Clean Fixed Method

You can streamline `determine_winner` by only defining the winning matchups for the player, defaulting to `"CPU"` if it's not a tie:

```
    def determine_winner(self, player, cpu):
        if player == cpu:
            return "TIE"

        winning_matchups = {
            "ROCK": "SCISSORS",
            "SCISSORS": "PAPER",
            "PAPER": "ROCK",
        }

        return "PLAYER" if winning_matchups.get(player) == cpu else "CPU"
```

Full Corrected Script

```
import random
import pygame
from game.button import ChoiceButton


class GameEngine:
```

```
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.choices = ["ROCK", "PAPER", "SCISSORS"]
        btn_w, btn_h = 130, 50
        gap = 20
        total_w = 3 * btn_w + 2 * gap
        start_x = (width - total_w) // 2
        btn_y = height - 85

        self.buttons = [
            ChoiceButton("ROCK", pygame.Rect(start_x, btn_y, btn_w, btn_h), (160, 50, 50), (200, 70, 70)),
            ChoiceButton("PAPER", pygame.Rect(start_x + btn_w + gap, btn_y, btn_w, btn_h), (40, 100, 170), (60, 130,
            ChoiceButton("SCISSORS", pygame.Rect(start_x + 2 * (btn_w + gap), btn_y, btn_w, btn_h), (180, 140, 30),
        ]

        self.player_choice = None
        self.cpu_choice = None
        self.result_text = "Make your move!"
        self.result_color = (220, 225, 235)

        self.player_score = 0
        self.cpu_score = 0

        self.round_resolved_time = 0
        self.display_duration = 1800
        self.showing_result = False

        self.font_title = pygame.font.SysFont(None, 36)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_arena = pygame.font.SysFont(None, 32)

    def determine_winner(self, player, cpu):
        if player == cpu:
            return "TIE"

        winning_matchups = {
            "ROCK": "SCISSORS",
            "SCISSORS": "PAPER",
            "PAPER": "ROCK",
        }
        return "PLAYER" if winning_matchups.get(player) == cpu else "CPU"

    def play_round(self, choice):
        self.player_choice = choice
        self.cpu_choice = random.choice(self.choices)

        outcome = self.determine_winner(self.player_choice, self.cpu_choice)
        if outcome == "PLAYER":
            self.player_score += 1
            self.result_text = f"You Win! {self.player_choice} beats {self.cpu_choice}."
            self.result_color = (80, 230, 120)
        elif outcome == "CPU":
            self.cpu_score += 1
            self.result_text = f"You Lose! {self.cpu_choice} beats {self.player_choice}."
            self.result_color = (240, 80, 80)
        else:
            self.result_text = f"It's a Draw! Both picked {self.player_choice}."
            self.result_color = (240, 210, 80)

        self.showing_result = True
        self.round_resolved_time = pygame.time.get_ticks()

    def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            for btn in self.buttons:
                if btn.contains(event.pos):
                    self.play_round(btn.choice_name)
                    break

    def update(self):
        now = pygame.time.get_ticks()
        if self.showing_result and (now - self.round_resolved_time >= self.display_duration):
            self.player_choice = None
            self.cpu_choice = None
            self.result_text = "Make your move!"
            self.result_color = (190, 195, 205)
            self.showing_result = False
```

```
    def render(self, screen):
        screen.fill((24, 28, 36))

        title_surf = self.font_title.render("Rock Paper Scissors", True, (245, 245, 245))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 14))

        p_surf = self.font_hud.render(f"Player Score: {self.player_score}", True, (100, 180, 255))
        c_surf = self.font_hud.render(f"CPU Score: {self.cpu_score}", True, (255, 120, 120))
        screen.blit(p_surf, (35, 52))
        screen.blit(c_surf, (self.width - c_surf.get_width() - 35, 52))

        pygame.draw.line(screen, (45, 52, 66), (25, 82), (self.width - 25, 82), 2)

        p_str = self.player_choice if self.player_choice else "--"
        c_str = self.cpu_choice if self.cpu_choice else "--"

        arena_p = self.font_arena.render(f"Your Pick:  {p_str}", True, (225, 225, 230))
        arena_c = self.font_arena.render(f"CPU Pick:  {c_str}", True, (225, 225, 230))
        screen.blit(arena_p, (self.width // 2 - arena_p.get_width() // 2, 115))
        screen.blit(arena_c, (self.width // 2 - arena_c.get_width() // 2, 155))

        res_surf = self.font_arena.render(self.result_text, True, self.result_color)
        screen.blit(res_surf, (self.width // 2 - res_surf.get_width() // 2, 205))

        for btn in self.buttons:
            btn.render(screen)
```

User prompt: Now, update the fixed script to include the following required features Task 2: Implement "First to X Wins" match victory state The game currently loops endlessly with no decisive match conclusion. Introduce a target win ceiling that halts play when reached, presents an end-of-match victory banner declaring the ultimate winner, and prompts the player to restart. Task 3: Implement adaptive AI pattern tracking The computer currently picks its moves entirely at random. Introduce adaptive intelligence that records the player's recent throw tendencies and dynamically biases CPU selections toward the counter-move when a player repeatedly favors a specific option. Task 4: Implement Procedural Gesture Icons & Reveal Animations Selections are currently rendered as plain text labels. Create distinct visual graphic icons for Rock, Paper, and Scissors, adding a synchronized countdown or shaking reveal animation before displaying both choices each turn. Provide the complete updated script.

Response:
```
import math
import random
import pygame
from game.button import ChoiceButton


class GameEngine:
    def __init__(self, width, height, target_wins=3):
        self.width = width
        self.height = height
        self.target_wins = target_wins

        self.choices = ["ROCK", "PAPER", "SCISSORS"]
        self.counter_moves = {
            "ROCK": "PAPER",
            "PAPER": "SCISSORS",
            "SCISSORS": "ROCK",
        }

        # Button Layout
        btn_w, btn_h = 130, 50
        gap = 20
        total_w = 3 * btn_w + 2 * gap
        start_x = (width - total_w) // 2
        btn_y = height - 85

        self.buttons = [
            ChoiceButton("ROCK", pygame.Rect(start_x, btn_y, btn_w, btn_h), (160, 50, 50), (200, 70, 70)),
            ChoiceButton("PAPER", pygame.Rect(start_x + btn_w + gap, btn_y, btn_w, btn_h), (40, 100, 170), (60, 130,
            ChoiceButton("SCISSORS", pygame.Rect(start_x + 2 * (btn_w + gap), btn_y, btn_w, btn_h), (180, 140, 30),
        ]
        self.restart_rect = pygame.Rect(width // 2 - 110, height - 90, 220, 55)

        # Game State
        self.player_choice = None
        self.cpu_choice = None
        self.player_score = 0
        self.cpu_score = 0
        self.game_over = False
        self.winner = None
```

```
        # AI History Tracker
        self.player_history = []
        self.history_window = 6

        # Animation & Phasing State
        # States: "IDLE", "ANIMATING", "RESULT"
        self.state = "IDLE"
        self.anim_start_time = 0
        self.anim_duration = 1800  # 3 shakes/beats
        self.round_resolved_time = 0
        self.display_duration = 2000

        self.result_text = "Make your move!"
        self.result_color = (220, 225, 235)

        # Fonts
        self.font_title = pygame.font.SysFont(None, 34)
        self.font_hud = pygame.font.SysFont(None, 24)
        self.font_arena = pygame.font.SysFont(None, 28)
        self.font_banner = pygame.font.SysFont(None, 44, bold=True)
        self.font_btn = pygame.font.SysFont(None, 26, bold=True)

    def determine_winner(self, player, cpu):
        if player == cpu:
            return "TIE"
        winning_matchups = {
            "ROCK": "SCISSORS",
            "SCISSORS": "PAPER",
            "PAPER": "ROCK",
        }
        return "PLAYER" if winning_matchups.get(player) == cpu else "CPU"

    def predict_cpu_move(self):
        """Adaptive AI: Weighs options using player history with weighted counters."""
        if len(self.player_history) < 2:
            return random.choice(self.choices)

        recent = self.player_history[-self.history_window:]
        counts = {choice: recent.count(choice) for choice in self.choices}
        total = len(recent)

        # Dynamic counter weighting: higher weight against the player's favored choice
        weights = []
        for choice in self.choices:
            # How likely is this choice to beat the player's anticipated moves?
            # choice beats whatever player move it counters
            countered_player_move = [k for k, v in self.counter_moves.items() if v == choice][0]
            player_prop = counts[countered_player_move] / total
            # Base probability + biased incentive proportional to player tendency
            weight = 0.2 + (0.8 * player_prop)
            weights.append(weight)

        return random.choices(self.choices, weights=weights, k=1)[0]

    def trigger_round(self, player_pick):
        if self.state != "IDLE" or self.game_over:
            return

        self.player_choice = player_pick
        self.cpu_choice = self.predict_cpu_move()
        self.player_history.append(player_pick)

        self.state = "ANIMATING"
        self.anim_start_time = pygame.time.get_ticks()

    def resolve_round(self):
        outcome = self.determine_winner(self.player_choice, self.cpu_choice)
        if outcome == "PLAYER":
            self.player_score += 1
            self.result_text = f"Round Won! {self.player_choice} beats {self.cpu_choice}."
            self.result_color = (80, 230, 120)
        elif outcome == "CPU":
            self.cpu_score += 1
            self.result_text = f"Round Lost! {self.cpu_choice} beats {self.player_choice}."
            self.result_color = (240, 90, 90)
        else:
            self.result_text = f"Draw! Both chose {self.player_choice}."
            self.result_color = (240, 210, 80)
```

```
            # Check match victory condition
            if self.player_score >= self.target_wins:
                self.game_over = True
                self.winner = "PLAYER"
            elif self.cpu_score >= self.target_wins:
                self.game_over = True
                self.winner = "CPU"

            self.state = "RESULT"
            self.round_resolved_time = pygame.time.get_ticks()

        def reset_game(self):
            self.player_score = 0
            self.cpu_score = 0
            self.game_over = False
            self.winner = None
            self.player_choice = None
            self.cpu_choice = None
            self.player_history.clear()
            self.result_text = "Make your move!"
            self.result_color = (220, 225, 235)
            self.state = "IDLE"

        def handle_event(self, event):
            if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                if self.game_over:
                    if self.restart_rect.collidepoint(event.pos):
                        self.reset_game()
                    return

                if self.state == "IDLE":
                    for btn in self.buttons:
                        if btn.contains(event.pos):
                            self.trigger_round(btn.choice_name)
                            break

        def update(self):
            now = pygame.time.get_ticks()

            if self.state == "ANIMATING":
                if now - self.anim_start_time >= self.anim_duration:
                    self.resolve_round()

            elif self.state == "RESULT" and not self.game_over:
                if now - self.round_resolved_time >= self.display_duration:
                    self.player_choice = None
                    self.cpu_choice = None
                    self.result_text = "Make your move!"
                    self.result_color = (190, 195, 205)
                    self.state = "IDLE"

        # --- Procedural Gesture Drawing ---
        def draw_gesture_icon(self, surface, gesture, center, scale=1.0, flip=False):
            cx, cy = center

            if gesture == "ROCK":
                color = (185, 75, 75)
                # Palm base
                pygame.draw.circle(surface, color, (cx, cy + int(4 * scale)), int(22 * scale))
                # Curled finger knuckles
                for offset_x in [-14, -5, 5, 14]:
                    pygame.draw.circle(surface, (215, 95, 95), (cx + int(offset_x * scale), cy - int(8 * scale)), int(8
                # Thumb tucked across
                thumb_dir = -1 if flip else 1
                pygame.draw.ellipse(surface, (160, 60, 60), (cx + int((thumb_dir * 4 - 8) * scale), cy + int(4 * scale),

            elif gesture == "PAPER":
                color = (65, 135, 225)
                # Open palm
                pygame.draw.circle(surface, color, (cx, cy + int(10 * scale)), int(18 * scale))
                # Extended vertical fingers
                finger_offsets = [(-15, 30), (-5, 36), (5, 35), (15, 28)]
                for off_x, height in finger_offsets:
                    finger_rect = pygame.Rect(cx + int((off_x - 4) * scale), cy - int(height * scale), int(8 * scale), i
                    pygame.draw.rect(surface, (95, 165, 245), finger_rect, border_radius=int(4 * scale))
                # Extended thumb
                thumb_x = cx + int((20 if not flip else -26) * scale)
                pygame.draw.ellipse(surface, (50, 115, 200), (thumb_x, cy - int(2 * scale), int(12 * scale), int(20 * sc
```

```
        elif gesture == "SCISSORS":
            color = (220, 180, 45)
            # Fist base
            pygame.draw.circle(surface, color, (cx, cy + int(12 * scale)), int(18 * scale))
            # Thumb & folded fingers
            pygame.draw.circle(surface, (180, 145, 35), (cx - int(7 * scale), cy + int(10 * scale)), int(9 * scale))
            pygame.draw.circle(surface, (180, 145, 35), (cx + int(7 * scale), cy + int(10 * scale)), int(9 * scale))
            # Two extended blades/fingers angled in a V
            f1_end = (cx - int(14 * scale), cy - int(32 * scale))
            f2_end = (cx + int(14 * scale), cy - int(32 * scale))
            pygame.draw.line(surface, (250, 215, 80), (cx - int(4 * scale), cy + int(2 * scale)), f1_end, int(8 * sc
            pygame.draw.line(surface, (250, 215, 80), (cx + int(4 * scale), cy + int(2 * scale)), f2_end, int(8 * sc

    def render(self, screen):
        screen.fill((22, 26, 35))

        # Header HUD
        title_surf = self.font_title.render("Rock Paper Scissors", True, (245, 245, 245))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 12))

        target_surf = self.font_hud.render(f"First to {self.target_wins} Wins", True, (160, 170, 190))
        screen.blit(target_surf, (self.width // 2 - target_surf.get_width() // 2, 45))

        p_surf = self.font_hud.render(f"Player: {self.player_score}/{self.target_wins}", True, (100, 185, 255))
        c_surf = self.font_hud.render(f"CPU: {self.cpu_score}/{self.target_wins}", True, (255, 115, 115))
        screen.blit(p_surf, (35, 45))
        screen.blit(c_surf, (self.width - c_surf.get_width() - 35, 45))

        pygame.draw.line(screen, (45, 52, 66), (25, 75), (self.width - 25, 75), 2)

        # Arena Display Positions
        player_arena_pos = (self.width // 4, 160)
        cpu_arena_pos = (3 * self.width // 4, 160)

        # Animation vs Result Visualization
        if self.state == "ANIMATING":
            now = pygame.time.get_ticks()
            elapsed = now - self.anim_start_time

            # 3 shakes/beats cadence (0 to 3)
            beat_progress = (elapsed / self.anim_duration) * 3
            beat_num = min(int(beat_progress) + 1, 3)
            shake_offset = int(math.sin(beat_progress * math.pi * 2) * 22)

            # Draw procedural animated fists bouncing in unison
            p_pos = (player_arena_pos[0], player_arena_pos[1] + shake_offset)
            c_pos = (cpu_arena_pos[0], cpu_arena_pos[1] + shake_offset)
            self.draw_gesture_icon(screen, "ROCK", p_pos, scale=1.3, flip=False)
            self.draw_gesture_icon(screen, "ROCK", c_pos, scale=1.3, flip=True)

            # Countdown prompt
            callouts = ["ROCK...", "PAPER...", "SCISSORS... SHOOT!"]
            callout = callouts[beat_num - 1]
            c_surf = self.font_arena.render(callout, True, (240, 210, 80))
            screen.blit(c_surf, (self.width // 2 - c_surf.get_width() // 2, 230))

        elif self.state == "RESULT" or self.game_over:
            # Reveal actual icons
            if self.player_choice:
                self.draw_gesture_icon(screen, self.player_choice, player_arena_pos, scale=1.3, flip=False)
            if self.cpu_choice:
                self.draw_gesture_icon(screen, self.cpu_choice, cpu_arena_pos, scale=1.3, flip=True)

            res_surf = self.font_arena.render(self.result_text, True, self.result_color)
            screen.blit(res_surf, (self.width // 2 - res_surf.get_width() // 2, 230))

        else:
            # Idle placeholder cues
            pygame.draw.circle(screen, (35, 42, 54), player_arena_pos, 38, 2)
            pygame.draw.circle(screen, (35, 42, 54), cpu_arena_pos, 38, 2)
            res_surf = self.font_arena.render(self.result_text, True, self.result_color)
            screen.blit(res_surf, (self.width // 2 - res_surf.get_width() // 2, 230))

        # Labels below arenas
        lbl_p = self.font_hud.render("YOU", True, (150, 160, 180))
        lbl_c = self.font_hud.render("CPU", True, (150, 160, 180))
        screen.blit(lbl_p, (player_arena_pos[0] - lbl_p.get_width() // 2, 100))
        screen.blit(lbl_c, (cpu_arena_pos[0] - lbl_c.get_width() // 2, 100))
```

```
        # Game Over Banner & Restart Button
        if self.game_over:
            overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
            overlay.fill((10, 12, 18, 175))
            screen.blit(overlay, (0, 0))

            if self.winner == "PLAYER":
                msg, color = "VICTORY! YOU WON THE MATCH!", (80, 240, 130)
            else:
                msg, color = "DEFEAT! CPU WON THE MATCH!", (250, 80, 80)

            banner = self.font_banner.render(msg, True, color)
            screen.blit(banner, (self.width // 2 - banner.get_width() // 2, self.height // 2 - 40))

            # Draw Play Again button
            mouse_pos = pygame.mouse.get_pos()
            btn_color = (60, 140, 220) if self.restart_rect.collidepoint(mouse_pos) else (40, 100, 180)
            pygame.draw.rect(screen, btn_color, self.restart_rect, border_radius=8)
            btn_txt = self.font_btn.render("PLAY AGAIN", True, (255, 255, 255))
            screen.blit(btn_txt, (self.restart_rect.centerx - btn_txt.get_width() // 2, self.restart_rect.centery -

        elif self.state != "ANIMATING":
            # Draw player choice buttons
            for btn in self.buttons:
                btn.render(screen)
```
